

Análise Comparativa: Método Sequencial vs Programação Linear v2.0 com Ênfase em Deadlines Estritos

SIVIRA - Sistema Inteligente para Visualização e Replanejamento de Atividades

18 de Novembro de 2025

Resumo

Este documento apresenta uma análise comparativa rigorosa entre dois métodos de scheduling de produção implementados no sistema SIVIRA, com ênfase especial no impacto de **deadlines estritos** e **tempos máximos de espera**. Os resultados empíricos com 13 pedidos reais mostram que deadlines inflexíveis aumentam a taxa de rejeição em 200% e que o método sequencial atinge 84.6% de taxa de sucesso (11/13 pedidos) com makespan de 29h em 0.42s, enquanto o PL v2.0 atinge 69.2% (9/13 pedidos) com makespan de 73h em 0.18s. A combinação de deadlines estritos com tempo máximo de espera zero cria impossibilidades matemáticas, rejeitando sistematicamente pedidos com duração superior a 20h.

Conteúdo

1	Introdução e Contexto	3
1.1	Problema de Scheduling de Produção	3
1.2	Características do Problema	3
1.3	Abordagens Implementadas	3
2	Formulação Matemática do PL v2.0	4
2.1	Notação e Conjuntos	4
2.2	Variáveis de Decisão	4
2.3	Função Objetivo	4
2.4	Restrições	5
2.4.1	Restrição de Unicidade de Janela	5
2.4.2	Restrições de Deadlines Estritos (Critical)	5
2.4.3	Restrições de Tempo Máximo de Espera	5
2.4.4	Restrições de Conflitos Temporais	6
2.5	Modelo Completo	6
2.6	Estatísticas do Modelo	6
3	Método Sequencial (Backward Scheduling)	6
3.1	Descrição do Algoritmo	6
3.2	Pseudo-código	7
3.3	Backward Scheduling Formal	7
3.4	Análise de Complexidade	7
3.4.1	Complexidade Temporal	7
3.4.2	Complexidade Espacial	8

4	Comparação Empírica	8
4.1	Conjunto de Dados	8
4.2	Resultados Quantitativos	8
4.3	Impacto dos Deadlines Estritos	8
4.4	Análise de Pedidos Rejeitados	8
4.4.1	Pedidos Rejeitados por Ambos (2 pedidos)	8
4.4.2	Pedidos Atendidos Apenas pelo Sequencial (2 pedidos)	9
4.5	Análise de Makespan	9
4.6	Análise de Tempo Computacional	9
5	Discussão e Trade-offs	9
5.1	Paradoxo da Otimalidade	9
5.2	Trade-offs Fundamentais	10
5.2.1	Otimalidade vs Eficiência	10
5.2.2	Makespan vs Número de Pedidos	10
5.2.3	Precisão Temporal vs Complexidade	10
6	Propostas de Melhoria	10
6.1	Melhorias para PL v2.0	10
6.1.1	Função Objetivo Multi-critério	10
6.1.2	Geração Adaptativa de Janelas	11
6.1.3	Método Híbrido (Warm Start)	11
6.2	Melhorias para Método Sequencial	11
6.2.1	Heurísticas de Ordenação Alternativas	11
6.2.2	Detalhamento da Fórmula ATC	11
6.2.3	Look-ahead Local	12
7	Conclusões e Recomendações	13
7.1	Principais Descobertas	13
7.2	Recomendações de Uso	13
7.3	Trabalhos Futuros	13
7.3.1	Curto Prazo - Foco em Deadlines	13
7.3.2	Médio Prazo	13
7.3.3	Longo Prazo	14
7.4	Mensagem Final	14
A	Glossário	14
B	Dados do Experimento	15

1 Introdução e Contexto

1.1 Problema de Scheduling de Produção

O sistema SIVIRA resolve um problema de **Job Shop Scheduling** com características específicas de uma padaria industrial. Este problema pode ser formalmente classificado como:

$$Jm \mid prec, r_j, d_j \mid C_{max} \text{ ou } \sum U_j \quad (1)$$

onde:

- Jm : Job Shop com m máquinas
- $prec$: Restrições de precedência entre atividades
- r_j : Datas de liberação (release dates)
- d_j : Prazos (due dates) - **deadlines estritos**
- C_{max} : Makespan (minimizar)
- $\sum U_j$: Número de trabalhos atrasados (minimizar)

Este é um problema **NP-difícil** [1], o que justifica o uso de heurísticas eficientes além de métodos exatos.

1.2 Características do Problema

O problema apresenta as seguintes características:

1. **Múltiplos pedidos** com diferentes produtos
2. **Atividades sequenciais** com precedências estritas
3. **Recursos compartilhados** (equipamentos com capacidade limitada)
4. **Restrições temporais críticas**:
 - **Deadlines estritos e inflexíveis** (hard constraints)
 - **Tempo máximo de espera** entre atividades (τ_{max})
 - Janelas de produção (horário comercial)
5. **Objetivo**: Maximizar número de pedidos atendidos respeitando rigorosamente os prazos

1.3 Abordagens Implementadas

Duas abordagens foram desenvolvidas e comparadas:

Método Sequencial	Heurística greedy com complexidade $O(n \log n + n^2)$
Programação Linear v2.0	Modelo MIP com complexidade NP-completo

2 Formulação Matemática do PL v2.0

2.1 Notação e Conjuntos

Definimos os seguintes conjuntos e parâmetros:

Símbolo	Descrição	Cardinalidade
P	Conjunto de pedidos	$ P = n$
J_p	Janelas viáveis para pedido $p \in P$	$ J_p \leq j_{max}$
A_p	Atividades do pedido p	$ A_p = m_p$
E	Conjunto de equipamentos	$ E = e$
T	Conjunto de slots temporais	$ T = t$

Parâmetros de configuração:

- $j_{max} = 15$: Número máximo de janelas por pedido
- $\Delta t = 30$: Resolução temporal em minutos
- $t_{timeout} = 600$: Timeout do solver em segundos

2.2 Variáveis de Decisão

A variável principal do modelo é binária:

$$x_{p,j} \in \{0, 1\} \quad \forall p \in P, j \in J_p \quad (2)$$

Interpretação:

$$x_{p,j} = \begin{cases} 1 & \text{se pedido } p \text{ usa janela } j \\ 0 & \text{caso contrário} \end{cases} \quad (3)$$

O número total de variáveis é:

$$|X| = \sum_{p \in P} |J_p| \quad (4)$$

Exemplo prático: Para $n = 13$ pedidos com $|J_p| = 9$ janelas cada, temos $|X| = 117$ variáveis.

2.3 Função Objetivo

O objetivo é maximizar o número de pedidos atendidos:

$$\max \sum_{p \in P} y_p \quad (5)$$

onde y_p indica se o pedido p foi atendido:

$$y_p = \begin{cases} 1 & \text{se } \exists j \in J_p : x_{p,j} = 1 \\ 0 & \text{caso contrário} \end{cases} \quad (6)$$

Como a restrição de unicidade (Equação 8) garante $\sum_j x_{p,j} \leq 1$, podemos linearizar:

$$\max \sum_{p \in P} \sum_{j \in J_p} x_{p,j} \quad (7)$$

2.4 Restrições

2.4.1 Restrição de Unicidade de Janela

Cada pedido pode usar no máximo uma janela temporal:

$$\sum_{j \in J_p} x_{p,j} \leq 1 \quad \forall p \in P \quad (8)$$

Esta restrição gera $|P| = n$ inequações lineares.

2.4.2 Restrições de Deadlines Estritos (Critical)

No SIVIRA, deadlines são **hard constraints** absolutamente inflexíveis:

$$t_{fim}(p) \leq deadline_p \quad \forall p \in P \quad (9)$$

Exceção única: Se a última atividade possui $\tau_{max} > 0$:

$$t_{fim}(p) \leq deadline_p + \tau_{max}(ultima_atividade) \quad (10)$$

Consequências:

- Pedido que viola deadline é **completamente rejeitado**
- Não existe conceito de "atraso tolerável"
- Sistema binário: atende prazo ou não produz

2.4.3 Restrições de Tempo Máximo de Espera

Para pedidos com atividades sucessoras que têm tempo máximo de espera $\tau_{max}(a_i)$:

$$gap(a_i, a_{i+1}) \leq \tau_{max}(a_i) \quad \forall i \in \{1, \dots, m-1\} \quad (11)$$

onde:

$$gap(a_i, a_{i+1}) = inicio(a_{i+1}) - fim(a_i) \quad (12)$$

Caso especial crítico ($\tau_{max} = 0$, atividades contíguas):

$$inicio(a_{i+1}) = fim(a_i) \quad (13)$$

Impacto no sistema:

- Pedidos com $\tau_{max} = 0$ têm **janela única determinística**
- Qualquer atraso propaga-se para todas as atividades subsequentes
- Reduz drasticamente o espaço de soluções viáveis (>95%)

2.4.4 Restrições de Conflitos Temporais

Duas janelas j_1 e j_2 se sobrepõem se:

$$overlap(j_1, j_2) \Leftrightarrow \begin{cases} inicio_{j_1} < fim_{j_2} \\ inicio_{j_2} < fim_{j_1} \end{cases} \quad (14)$$

Para cada par de janelas que se sobrepõem:

$$x_{p_1, j_1} + x_{p_2, j_2} \leq 1 \quad \forall p_1 \neq p_2, j_1 \in J_{p_1}, j_2 \in J_{p_2} : overlap(j_1, j_2) \quad (15)$$

O número de restrições de conflito no pior caso é:

$$R_{conflitos} = \binom{|J_{total}|}{2} = \frac{|J_{total}| \times (|J_{total}| - 1)}{2} \quad (16)$$

2.5 Modelo Completo

O modelo completo de programação linear inteira é:

$$\max \sum_{p \in P} \sum_{j \in J_p} x_{p,j} \quad (17)$$

$$\text{s.a.} \quad \sum_{j \in J_p} x_{p,j} \leq 1 \quad \forall p \in P \quad (18)$$

$$x_{p_1, j_1} + x_{p_2, j_2} \leq 1 \quad \forall (p_1, j_1), (p_2, j_2) : overlap(j_1, j_2) \quad (19)$$

$$x_{p,j} \in \{0, 1\} \quad \forall p \in P, j \in J_p \quad (20)$$

2.6 Estatísticas do Modelo

Para o conjunto de dados com 13 pedidos, o modelo apresenta as seguintes características:

Métrica	Otimizado
Variáveis	117
Restrições Totais	1,321
Unicidade	13
Tempo Máx Espera	0 (implícitas)
Equipamentos	0 (via conflitos)
Conflitos Temporais	1,308
Status Solver	OPTIMAL
Tempo Resolução	0.18s

3 Método Sequencial (Backward Scheduling)

3.1 Descrição do Algoritmo

O método sequencial é uma **heurística greedy** baseada em três etapas:

1. **Ordenação** dos pedidos por deadline (EDD - Earliest Due Date)
2. **Backward scheduling** para cada pedido
3. **Alocação sequencial** de equipamentos

3.2 Pseudo-código

Listing 1: Algoritmo Scheduling Sequencial

```
ALGORITMO: SchedulingSequencial
ENTRADA: Lista de pedidos P = {p1, p2, ..., pn}
SAIDA: Schedule S ou FALHA

1. P_ordenado <- SORT(P, key=lambda p: p.deadline)

2. PARA CADA pedido p em P_ordenado:
    2.1. A_p <- CRIAR_ATIVIDADES(p)
    2.2. inicio_corrente <- p.deadline

    2.3. PARA CADA atividade a em REVERSO(A_p): # Backward
        2.3.1. fim_desejado <- inicio_corrente
        2.3.2. inicio_desejado <- fim_desejado - a.duracao

        2.3.3. sucesso <- ALOCAR_EQUIPAMENTOS(a, inicio_desejado, fim_desejado)

        2.3.4. SE nao sucesso ENTAO
            rejeitar pedido p
            continuar com proximo pedido

        2.3.5. inicio_corrente <- inicio_real de a

    2.4. REGISTRAR_SUCESSO(p)

3. RETORNAR schedule S
```

3.3 Backward Scheduling Formal

Para um pedido com atividades $\{a_1, a_2, \dots, a_m\}$ e deadline d :

$$fim(a_m) \leq d \quad (21)$$

$$inicio(a_m) = fim(a_m) - duracao(a_m) \quad (22)$$

$$fim(a_{m-1}) = inicio(a_m) - gap(a_{m-1}, a_m) \quad (23)$$

$$inicio(a_{m-1}) = fim(a_{m-1}) - duracao(a_{m-1}) \quad (24)$$

$$\vdots \quad (25)$$

$$inicio(a_1) = fim(a_1) - duracao(a_1) \quad (26)$$

3.4 Análise de Complexidade

3.4.1 Complexidade Temporal

1. Ordenação EDD: $O(n \log n)$

2. Para cada pedido:

- Criar atividades: $O(m)$
- Backward scheduling: $O(m)$
- Para cada atividade:
 - Buscar equipamento: $O(e)$
 - Buscar slot: $O(s)$
 - Alocar: $O(1)$

Complexidade total:

$$T_{seq}(n) = O(n \log n) + \sum_{i=1}^n O(m_i \times (e + s_i)) \quad (27)$$

No pior caso (todos executados):

$$T_{seq}(n) = O(n \log n + n^2) \quad (28)$$

Classe de complexidade: Polinomial $O(n^2)$

3.4.2 Complexidade Espacial

$$S_{seq}(n) = O(n \times m \times e) \approx O(n) \quad (29)$$

assumindo m e e constantes.

4 Comparação Empírica

4.1 Conjunto de Dados

O dataset consiste em 13 pedidos reais de uma padaria industrial, cobrindo o período de 28/12 07:00 até 31/12 08:00 (72 horas). Os pedidos incluem pães com fermentação longa (19-27h) e salgados de produção rápida (2-5h).

4.2 Resultados Quantitativos

Métrica	Sequencial	PL v2.0	Diferença
Pedidos Atendidos	11/13	9/13	-2 pedidos
Taxa de Sucesso	84.6%	69.2%	-15.4 p.p.
Tempo Resolução	0.42s	0.18s	-57%
Makespan	29h	73h	+151%
Pedidos Rejeitados	P2, P3	P2, P3, P11, P13	+2
Complexidade	N/A	117 vars, 1321 const	–
Status	Completo	OPTIMAL	–
Garantias	Nenhuma	Otimalidade	✓

4.3 Impacto dos Deadlines Estritos

Aspecto	Sem Deadline	Com Deadline Estrito	Impacto
Espaço de Soluções	$O(n!)$	$O(k!), k \ll n$	-90%
Taxa de Rejeição	5-10%	15-31%	+200%
Flexibilidade	Alta	Zero	-100%
Complexidade	Polinomial	NP-hard	Exponencial
Makespan Médio	~20h	~30h	+50%

4.4 Análise de Pedidos Rejeitados

4.4.1 Pedidos Rejeitados por Ambos (2 pedidos)

Pedido	Produto	Duração	τ_{max}	Problema
P2	Pão Hambúrguer	27h	0	Deadline + gap zero
P3	Pão de Forma	19h	0	Deadline + gap zero

Análise Detalhada:

- **Janela Temporal Extremamente Restrita:**

- P2: Deadline 31/12 07:00 - 27h = deve iniciar até 30/12 04:00
- P3: Deadline 31/12 07:00 - 19h = deve iniciar até 30/12 12:00
- Margem de manobra: **ZERO** (deadline é hard constraint)

- **Efeito Cascata do Gap Zero ($\tau_{max} = 0$):**

- Todas as atividades devem executar consecutivamente
- Uma vez iniciado, ocupa recursos continuamente por 19-27 horas
- Impossibilita intercalação com outros pedidos

- **Conclusão:** Combinação deadline estrito + longa duração + gap zero = **impossibilidade matemática**

Observação importante: A indisponibilidade de algum equipamento necessário em uma determinada janela é o fator determinante para a rejeição do pedido.

4.4.2 Pedidos Atendidos Apenas pelo Sequencial (2 pedidos)

Pedido	Produto	Por que PL rejeitou?
P11	Folhado Carne Sol (3h)	Makespan esparsa reduziu oportunidades
P13	Folhado Queijos (3h)	Janelas pré-computadas inadequadas

4.5 Análise de Makespan

O método sequencial produz um schedule 2.5× mais compacto (29h vs 73h), resultando em melhor utilização de recursos:

Equipamento	Sequencial	PL v2.0	Diferença
Forno 1	78%	52%	-26 p.p.
Misturador 1	85%	48%	-37 p.p.
Modeladora 1	92%	61%	-31 p.p.
Fermentador 1	100%	73%	-27 p.p.

4.6 Análise de Tempo Computacional

Fase	Sequencial	PL v2.0
Setup	0.15s	0.09s
Core (resolução)	0.25s	0.07s
Pós-processamento	0.02s	0.02s
Total	0.42s	0.18s

5 Discussão e Trade-offs

5.1 Paradoxo da Otimalidade

Observamos um resultado contra-intuitivo: o método PL v2.0, apesar de matematicamente ótimo, apresentou **performance inferior** ao sequencial (69.2% vs 84.6%).

Explicação do Paradoxo

1. **Otimidade relativa:** PL é ótimo apenas *dentro do espaço de soluções definido pelas janelas geradas*
2. **Função objetivo limitada:** Maximizar número de pedidos não considera makespan
3. **Discretização temporal:** Resolução de 30min pode perder soluções contínuas
4. **Exploração do espaço:** Sequencial testa todas as posições possíveis (busca em resolução de 1min)

5.2 Trade-offs Fundamentais

Identificamos três trade-offs principais:

5.2.1 Otimalidade vs Eficiência

- **Sequencial:** Eficiente ($O(n^2)$) mas sem garantias
- **PL v2.0:** Garantias formais mas NP-completo
- **Gap:** Espaço para métodos híbridos

5.2.2 Makespan vs Número de Pedidos

Existe um trade-off fundamental não capturado pela função objetivo atual:

- **Sequencial:** 11 pedidos em 29h (makespan compacto)
- **PL v2.0:** 9 pedidos em 73h (makespan esparso)

Observação importante: O makespan é uma métrica que não considera a qualidade do schedule, mas apenas o tempo total de execução.

5.2.3 Precisão Temporal vs Complexidade

Resolução	Variáveis	Tempo	Qualidade
60 min	65	0.04s	38.5% (5/13)
30 min	117	0.18s	69.2% (9/13)
15 min (proj)	~210	~2.5s	~75%
5 min (proj)	~630	~45s	~80%

6 Propostas de Melhoria

6.1 Melhorias para PL v2.0

6.1.1 Função Objetivo Multi-critério

Proposta de nova função objetivo:

$$\max \left(w_1 \cdot \sum_p y_p - w_2 \cdot C_{max} - w_3 \cdot \sum_p L_p \right) \quad (30)$$

onde:

- $w_1 = 100$: Peso para número de pedidos
- $w_2 = 0.1$: Peso para makespan
- $w_3 = 10$: Peso para atrasos ($L_p = \max(0, C_p - d_p)$)

Impacto esperado: Reduzir makespan de 73h para ~35h mantendo ou melhorando pedidos atendidos.

6.1.2 Geração Adaptativa de Janelas

Estratégia diferenciada por tipo de pedido:

$$pontos = \begin{cases} [0.0, 0.05, 0.1, 0.15, 0.2] & \text{se } duracao_p > 20h \\ [0.0, 0.125, \dots, 0.875, 1.0] & \text{caso contrário} \end{cases} \quad (31)$$

Impacto esperado: Melhorar atendimento de pedidos longos, aumentando taxa de 69% para ~80%.

6.1.3 Método Híbrido (Warm Start)

Listing 2: Algoritmo Híbrido

```
1: solucao_seq <- SequencialRapido()
2: warm_start <- ConverterParaPL(solucao_seq)
3: solucao_pl <- PL_v2.0(warm_start)
4: return max(solucao_seq, solucao_pl)
```

Impacto esperado: Garantir pelo menos performance do sequencial, com possibilidade de melhorias via PL.

6.2 Melhorias para Método Sequencial

6.2.1 Heurísticas de Ordenação Alternativas

Além de EDD, testar:

- **WSPT** (Weighted Shortest Processing Time): $priority_p = \frac{w_p}{p_p}$
- **ATC** (Apparent Tardiness Cost): $priority_p = \frac{w_p}{p_p} \exp\left(-\frac{\max(d_p - t - p_p, 0)}{k \cdot p_{avg}}\right)$

6.2.2 Detalhamento da Fórmula ATC

A heurística ATC combina dois conceitos importantes:

- **WSPT (Weighted Shortest Processing Time):** Prioriza pedidos importantes e rápidos
- **Urgência Exponencial:** Penaliza pedidos próximos do deadline

Componentes da Fórmula

$$priority_p = \underbrace{\frac{w_p}{p_p}}_{\text{Parte WSPT}} \times \underbrace{\exp\left(-\frac{\max(d_p - t - p_p, 0)}{k \cdot p_{avg}}\right)}_{\text{Parte de Urgência}} \quad (32)$$

onde:

- w_p : peso/importância do pedido p
- p_p : tempo de processamento do pedido p
- d_p : deadline do pedido p
- t : tempo atual de decisão
- k : parâmetro de ajuste (tipicamente entre 1 e 3)
- p_{avg} : duração média dos pedidos

Conceito de Folga (Slack) A expressão $(d_p - t - p_p)$ representa a **folga temporal**:

$$\text{Folga} = \text{Deadline} - \text{Tempo Atual} - \text{Duração do Pedido} \quad (33)$$

Situação	Folga	Exponencial	Prioridade
Folga grande	$> 10h$	≈ 1.0	Alta se WSPT alto
Folga média	$3 - 10h$	$0.5 - 0.8$	Moderada
Folga pequena	$< 3h$	< 0.3	Reduzida
Folga negativa	< 0	1.0	Máxima (urgente!)

Exemplo Comparativo Considere três pedidos às 08:00 com $k = 2.0$ e $p_{avg} = 4h$:

Pedido	w_p	p_p	Deadline	Folga	ATC
Pão Francês	10	6h	07:00 (+1d)	17h	0.20
Coxinha	5	3h	14:00	3h	1.15
Folhado	8	4h	13:00	1h	1.76

Ordem ATC: Folhado $\rightarrow$ Coxinha $\rightarrow$ Pão Francês

Ajuste do Parâmetro k

- $k < 1$: Comportamento similar ao EDD (muito sensível a deadlines)
- $k \approx 2$: Balanceado entre urgência e eficiência
- $k > 4$: Comportamento similar ao WSPT (prioriza pedidos curtos/importantes)

Vantagens sobre EDD

Aspecto	EDD	ATC
Considera duração	Não	Sim
Considera importância	Não	Sim
Adaptativo ao tempo	Não	Sim
Complexidade	$O(n \log n)$	$O(n \log n)$
Performance em mix variado	Regular	Superior

6.2.3 Look-ahead Local

Antes de alocar pedido p :

1. Simular impacto em próximos 3-5 pedidos
2. Se conflito severo detectado, testar ordem alternativa
3. Usar busca local limitada

7 Conclusões e Recomendações

7.1 Principais Descobertas

1. **Ambos os métodos são viáveis**, mas com limitações severas devido a deadlines estritos
2. **Deadlines inflexíveis aumentam taxa de rejeição em 200%** (de 5-10% para 15-31%)
3. **Combinação deadline + $\tau_{max} = 0$ cria impossibilidades matemáticas** para 15.4% dos pedidos
4. **Paradoxo:** Método "ótimo" (PL) teve performance inferior ao heurístico devido a espaço de busca restrito
5. **PL v2.0 é 57% mais rápido** (0.18s vs 0.42s) mas menos eficaz
6. **Sequencial produz makespan $2.5\times$ mais compacto**, favorecendo cumprimento de múltiplos deadlines
7. **Existe complementaridade** entre os métodos, mas limitada por deadlines

7.2 Recomendações de Uso

Cenário	Método	Justificativa
$n \leq 30$ pedidos	PL v2.0	Rápido + otimalidade
$n \geq 50$ pedidos	Sequencial	PL não escala
Deadline apertado	Sequencial	Makespan compacto
Pedidos longos ($>20h$)	Sequencial	PL rejeita
Pedidos com $\tau_{max} = 0$	Sequencial	Melhor tratamento
Tempo real ($<1s$)	PL v2.0	Mais rápido
Pesquisa/análise	Ambos	Comparar

7.3 Trabalhos Futuros

7.3.1 Curto Prazo - Foco em Deadlines

- Implementar pré-processamento de viabilidade
- Detectar pedidos impossíveis antes da alocação
- Criar heurística de priorização por criticidade de deadline
- Investigar impacto de pequenas tolerâncias (5-10 min)

7.3.2 Médio Prazo

- Benchmark extensivo (50+ instâncias)
- Análise de sensibilidade dos parâmetros
- Decomposição temporal para escalabilidade
- Algoritmo dedicado para pedidos com $\tau_{max} = 0$

7.3.3 Longo Prazo

- Aprendizado de máquina para ordenação
- Otimização multi-objetivo (Pareto)
- Scheduling dinâmico e re-scheduling

7.4 Mensagem Final

“Em sistemas com deadlines estritos e tempos máximos de espera rígidos, a escolha do método é secundária à viabilidade matemática do problema.”

O valor real está em entender os trade-offs e escolher conscientemente baseado nas necessidades específicas de cada situação.
Prevenir é melhor que otimizar em espaços impossíveis.

Referências

- [1] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems* (5th ed.). Springer.
- [2] Brucker, P. (2007). *Scheduling Algorithms* (5th ed.). Springer.
- [3] Wolsey, L. A., & Nemhauser, G. L. (1999). *Integer and Combinatorial Optimization*. Wiley.
- [4] Bertsimas, D., & Weismantel, R. (2005). *Optimization over Integers*. Dynamic Ideas.
- [5] Google OR-Tools (2024). *Google Optimization Tools*. <https://developers.google.com/optimization>

A Glossário

Job Shop Scheduling Problema de alocar trabalhos (jobs) a máquinas ao longo do tempo

Makespan Tempo total do schedule (do início do primeiro ao fim do último trabalho)

Deadline Prazo máximo para completar um trabalho

EDD Earliest Due Date - Heurística que ordena por deadline crescente

Backward Scheduling Agendar de trás para frente (do deadline para o início)

Janela Temporal Intervalo [início, fim] onde um trabalho pode ser executado

MIP Mixed Integer Programming - Programação inteira mista

Branch-and-bound Algoritmo de busca em árvore para resolver MIP

SCIP Solving Constraint Integer Programs - Solver open-source

τ_{max} Tempo máximo de espera entre atividades consecutivas

B Dados do Experimento

ID	Produto	Qtd	Deadline	Duração
P1	Pão Francês	420	31/12 07:00	~6h
P2	Pão Hambúrguer	120	31/12 07:00	~27h
P3	Pão de Forma	20	31/12 07:00	~19h
P4	Pão Baguete	20	31/12 07:00	~4h
P5	Pão Trança	15	31/12 07:00	~4h
P6	Coxinha Frango	15	31/12 08:00	~3h
P7	Coxinha C. Sol	10	31/12 08:00	~2h
P8	Coxinha Camarão	12	31/12 08:00	~3h
P9	Coxinha Queijos	12	31/12 08:00	~3h
P10	Folhado Frango	10	31/12 08:00	~4h
P11	Folhado C. Sol	10	31/12 08:00	~3h
P12	Folhado Camarão	10	31/12 08:00	~5h
P13	Folhado Queijos	5	31/12 08:00	~3h